

AmaliTech — Backend Engineering Track

Student Grade Management System

Engineering Deep-Dive & Interview Preparation Report

develop branch — v2.0.0

Console-based Java 17 application · Layered architecture · SOLID principles
Maven build · JUnit 5 + Mockito (377 tests) · JaCoCo + SonarQube · GitHub Actions CI/CD

Prepared for supervisor / engineering review

Table of Contents

Part I — Project Overview, Objectives & Features	3
Part II — Project Structure & Rationale	—
Part III — Object-Oriented Programming Concepts Applied	—
Part IV — SOLID Principles: Where and How	—
Part V — Java Language Mechanics (Behind the JVM Curtain)	—
Part VI — Logging Architecture	—
Part VII — Exception Handling Architecture	—
Part VIII — Class-by-Class Reference (all packages, all methods)	—
Part IX — Testing Strategy: JUnit 5, Mockito, and Alternatives	—
Part X — Git & GitHub Workflow	—
Part XI — Design Patterns Identified	—
Part XII — Known Limitations & Honest Discussion Points	—
Part XIII — Anticipated Interview Questions (from the assignment rubric)	—
Part XIV — Conclusion	—

(Page numbers are approximate — this document paginates naturally when printed/exported to PDF.)

Part I — Project Overview, Objectives & Features

1.1 What This Project Is

The Student Grade Management System is a console (terminal) Java application that lets a school manage students, subjects, and grades, and derive reports and analytics from that data: GPA on a 4.0 scale, class-wide statistics, bulk CSV grade import, exported text reports, and student search. It began as a smaller Lab 1 exercise (a flat, 9-class in-memory design) and was extended for this assessment ("v2") into a layered, testable, SOLID-compliant application with a real build (Maven), a real test suite (JUnit 5 + Mockito, 377 tests), a code-coverage and static-analysis toolchain (JaCoCo + SonarQube), and a real CI/CD pipeline (GitHub Actions).

There is no external database and no network layer. All data — students, subjects, grades — lives in fixed-size in-memory arrays for the lifetime of the process and is lost when the program exits. This was a deliberate constraint of the assignment brief, not an oversight, and is discussed candidly in Part XII.

1.2 Objectives (from the assignment brief)

The brief this project answers to (`ReadMe-v2.md`) states five explicit learning objectives. Each maps directly onto something concrete in this codebase:

Objective	Where it shows up in this codebase
Robust exception handling, JUnit/Mockito tests, SOLID	9 custom exceptions under one <code>ApplicationException</code> root (a 10th, dead, was found and removed); 377 tests; SOLID audit (Part IV)
Git workflows / branching strategies	16 feature branches merged into <code>develop</code> , never directly to <code>main</code> (Part X)
Evaluate code quality (static analysis, coverage)	JaCoCo coverage plugin wired into the Maven build (Part IX)
Debug complex bugs systematically	14 requirements-review issues found and fixed, each documented with root cause + fix in <code>CHANGELOG.md</code>
Comprehensive test suites with mocking, CI-integrated	Two-test-file-per-class convention (real + Mockito); GitHub Actions runs the suite on every push

1.3 The Menu & What Each Feature Does

The application exposes exactly ten menu options. Each is implemented as its own class under the `console` package (Part VIII explains why), and each was one of the six required "new" user stories in the brief, plus the four carried over from Lab 1.

#	Feature	What happens internally	Write or Read?
1	Add Student	Reads name/age/email/phone/type, constructs a <code>RegularStudent</code> or <code>HonorsStudent</code> , validates via <code>StudentValidator</code> , stores via <code>StudentService</code> → <code>StudentRepository</code>	Write
2	View Students	Lists every student with type, average, pass/fail status; hydrates each student's grade list from <code>GradeManager</code> first	Read
3	Record Grade	Picks a student, a subject (Core/Elective), a raw score 0–100; constructs a <code>Grade</code> (which self-validates in its constructor) and persists it	Write
4	View Grade Report	Prints one student's full grade history plus Core/Elective/overall averages	Read
5	Export Grade Report	Builds summary and/or detailed report text via <code>ReportGenerator</code> , writes it to <code>reports/*.txt</code> via <code>FileExporter</code>	Read + file write (no domain mutation)
6	Calculate Student GPA	Converts every grade to a 4.0-scale GPA point value and letter grade via <code>GPACalculator</code> / <code>GpaLetterGrade</code> , then computes cumulative GPA and class rank	Read
7	Bulk Import Grades	Parses a CSV under <code>imports/</code> via <code>CSVParser</code> , skips invalid rows (never aborts the whole file), writes every accepted row as a new <code>Grade</code> , and writes a log file of the run	Write
8	View Class Statistics	Grade distribution by letter, mean/median/mode/standard deviation, subject averages, Regular-vs-Honors comparison, all via <code>StatisticsCalculator</code>	Read
9	Search Students	By ID, partial name, grade range, or type, via <code>StudentSearcher</code> ; results can be viewed in full or exported	Read
10	Exit	Prints a goodbye message and terminates the menu loop	—

Role-based access (optional): on startup the user may enable role-based access and choose `TEACHER` or `STUDENT`. A Student can never reach options 1, 2, 3, or 7 — every write action is teacher-only, and as of the latest fix, those options don't even appear in a Student's menu (not just "denied after picking"). This is enforced twice: the menu itself filters by `MenuAction.isAuthorizedFor(Role)`, and the dispatch loop re-checks it before calling `execute()`, so typing an unlisted number directly still fails safely.

Part II — Project Structure & Rationale

2.1 The Package Layout

```
src/
├─ app/
│   ├── Main.java           composition root: builds the object graph, hands off to ConsoleApp
│   └── ConsoleApp.java      the menu loop itself - takes its Scanner via constructor, unlike the
                             old Main, so it's unit-testable with scripted input
├─ console/                 one class per menu feature (10 classes) + MenuAction interface
├─ model/
│   ├── student/            Student (abstract), RegularStudent, HonorsStudent
│   ├── subject/            Subject (abstract), CoreSubject, ElectiveSubject
│   ├── grade/              Grade, Gradable interface
│   └── enums/              Role, StudentType, StudentStatus, SubjectType, LetterGrade, GpaLetterGrade
├─ manager/                StudentManager, GradeManager (facades), StudentSearcher + Searchable
├─ service/                StudentService/Impl, GradeService/Impl - business rules
├─ repository/             student/, grade/, subject/ - each: interface + Impl, array-backed
├─ calculators/            GPACalculator + Calculable, StatisticsCalculator
├─ export/                 ReportGenerator + Exportable, FileExporter
├─ imports/                CSVParser, BulkImportService
├─ dto/ + mapper/          StudentDTO/Mapper, GradeDTO/Mapper - read-only display projections
├─ utils/                  InputSanitizer, DateFormats, validators/ (StudentValidator, SubjectValidator)
├─ logging/                Logger - dependency-free DEBUG/INFO/WARN/ERROR Logger
└─ exceptions/             ApplicationException (root) + 10 custom exceptions
```

2.2 Why It's Organized This Way — Layer by Layer

This is a classic **layered architecture**: each layer only talks to the layer directly beneath it, and only through that layer's interface, never its concrete class. Requests flow strictly downward; nothing in a lower layer ever calls back up into a higher one.

Layer	Depends on	Responsibility	Never does
model	nothing	What a Student/Subject/Grade is — pure data + domain rules that need no storage (e.g. <code>isPassing()</code>)	No <code>Scanner</code> , no <code>System.out</code> , no persistence knowledge
repository	model, exceptions	How an entity is stored/fetched (array-backed)	No business rules — a repository never asks "is this grade valid," only "does this ID exist"
service	model, utils.validators, repository (interfaces)	Business rules: does the student exist, is the grade in range, log the outcome	Never imports a repository's <code>Impl</code> class directly — only its interface
manager	service (interfaces)	Higher-level facade: hydrate a student's grades, compute class averages, format grade-history tables	No business rules of its own — delegates
calculators, export, imports	manager, model	One focused computation/IO concern each (GPA math, statistics, report text, file writing, CSV parsing)	Never touches <code>Scanner</code> / <code>console</code>
console	everything above	The only layer allowed to read <code>Scanner</code> input or print to <code>System.out</code>	No business logic of its own
Main	everything (composition root)	new s every concrete implementation exactly once and wires the object graph together	No business logic, no I/O formatting of its own

2.3 A Design Decision Worth Explaining: Colocated Interfaces

Every interface in this codebase sits in the *same package* as its one implementer — `service/StudentService.java` next to `service/StudentServiceImpl.java`, `calculators/Calculable.java` next to `calculators/GPACalculator.java`, and so on. This wasn't always true: early on there was a separate `interfaces/` package holding `Calculable`, `Exportable`, and `Searchable` while `service/` / `repository` already colocated theirs — two different conventions for the identical relationship. It was unified during a professional-structure review (see Part X's commit history) because a class implementing an interface isn't "a kind of interface" — it belongs with its own feature area, not filed away by which contract it happens to satisfy.

2.4 Why Fixed-Size Arrays, Not a HashMap or a Database

The assignment brief explicitly required in-memory, array-based storage (a carry-over constraint from the original Lab 1 spec). Each repository owns exactly one primitive array (`Student[50]`, `Grade[200]`, `Subject[50]`) plus a running count of how many slots are filled. A linear scan (`for` loop) is used for lookups — $O(n)$, not $O(1)$ like a `HashMap` would give — which is an accepted, deliberate trade-off given the array-size ceiling (at most 200 grades) makes the performance difference immaterial in practice.

Part III — Object-Oriented Programming Concepts Applied

3.1 Encapsulation

Every field in every domain class is `private`; access is only through explicit getters/setters or behavior methods. `Student.grades` (a `List<Double>`) is never exposed for direct external mutation except through `addGrade(double) / setGrade(double)` — callers cannot reach in and corrupt the list's internal state without going through a method that owns the invariant.

```
public abstract class Student {
    private String studentId;           // no caller can see or set this except through a method
    private List<Double> grades = new ArrayList<>();

    public List<Double> getGrades() { return grades; } // read access
    public void addGrade(double grade) { grades.add(grade); } // controlled write
}
```

Honest caveat: `getGrades()` returns the live, mutable `List` reference rather than an unmodifiable copy — a determined caller *could* call `.clear()` on the returned list from outside the class. In practice only `StudentManager.hydrateGrades()` does this (deliberately, to refresh the transient list from the real source of truth in `GradeRepository`), but a stricter version of encapsulation would return `Collections.unmodifiableList(grades)` or a defensive copy. Good to know as a discussion point if asked "is this fully encapsulated?"

3.2 Inheritance

Two parallel class hierarchies model "is-a" relationships:

Hierarchy	Base (abstract)	Subclasses	What differs per subclass
Students	Student	RegularStudent , HonorsStudent	Passing grade threshold (50% vs 60%), extra <code>checkHonorsEligibility()</code> behavior, own <code>StudentType</code> value
Subjects	Subject	CoreSubject , ElectiveSubject	Display text, <code>SubjectType</code> value, an <code>isMandatory()</code> flag

Shared state and behavior (student ID, name, age, grade list, `calculateAverageGrade()`, `isPassing()`) live once in the abstract base class; only what genuinely differs is overridden. This is the textbook argument for inheritance: avoid duplicating the 90% that's identical across subtypes.

3.3 Polymorphism

Every place that holds a `Student` reference — `StudentRepositoryImpl`'s array, `List<Student>` returned by `getAllStudents()`, the parameter of `displayStudentDetails()` — never needs to know or check whether the actual object is a `RegularStudent` or `HonorsStudent`. Calling `student.getPassingGrade()` or `student.displayStudentDetails()` runs whichever subclass's override actually applies, decided at runtime by the object's real type — not by the reference's declared type. Under the hood (see Part V) this is **dynamic dispatch**: the JVM looks up the method in the object's actual class's method table at the moment the call executes, not at compile time.

A second, narrower form of polymorphism used here is **pattern-matching instanceof** (Java 16+), used sparingly and only where genuinely subclass-specific behavior is needed with no base-class equivalent:

```
if (student instanceof HonorsStudent hs) {
    hs.checkHonorsEligibility(); // only HonorsStudent has this method
}
```

This is deliberately *not* used for classifying "which type is this student" (that would fight polymorphism) — see Part IV's OCP discussion for where that exact anti-pattern was found and fixed by introducing `Student.getType(): StudentType` instead.

3.4 Abstraction

Two abstraction mechanisms are used, deliberately for different purposes:

- **Abstract classes** (`Student`, `Subject`) — used when subclasses share real, inherited *state* (fields) as well as behavior. A subclass can only extend one abstract class.
- **Interfaces** (`Calculable`, `Exportable`, `Searchable`, `Gradable`, `MenuAction`, every `*Service / *Repository`) — used to describe a pure *contract* with no shared state, so a class can implement many of them at once and different, unrelated classes can satisfy the same contract (e.g. any future report format could implement `Exportable` without touching `ReportGenerator`).

Part IV — SOLID Principles: Where and How

The assignment brief names all five explicitly and even suggests example classes. What follows is where each principle is *actually* demonstrated in this codebase today, including two real violations that were found during a dedicated audit and fixed (not just theoretical box-ticking).

4.1 Single Responsibility Principle (SRP)

Definition: a class should have exactly one reason to change.

Class	Its one job
<code>GPA Calculator</code>	Percentage↔GPA↔letter conversion, cumulative GPA, class rank — nothing else
<code>Statistics Calculator</code>	Descriptive statistics over a grade list — nothing else
<code>ReportGenerator</code>	Builds report <i>text</i> — knows nothing about files
<code>FileExporter</code>	Writes a string to a file — knows nothing about report content
<code>CSVParser</code>	Parses CSV text into valid rows + error messages — never persists anything
<code>BulkImportService</code>	Orchestrates parse → persist → log for a bulk import — the actual parsing and persisting are delegated

SRP was violated, found, and fixed — twice: `Main.java` had grown to 760 lines doing composition-root wiring, console I/O, business orchestration for all 9 menu features, role authorization, and exception-to-message translation — five reasons to change in one file. Fixed by extracting one `console.*Action` class per feature, each implementing `MenuAction`. A second, smaller SRP violation then surfaced in what was left: `Main` still mixed "build the dependency graph" with "run the interactive menu loop," and the loop was bound to `System.in` in a `static final Scanner` field, which also made it untestable. Split again into `app.Main` (composition root only, ~65 lines) and `app.ConsoleApp` (the loop, taking its `Scanner` via constructor instead of a static field). This is a genuinely good interview talking point: it shows you can *find* an SRP violation in real code, not just recite the definition — and that fixing one can reveal a second, smaller one underneath.

4.2 Open/Closed Principle (OCP)

Definition: open for extension, closed for modification — adding new behavior shouldn't require editing existing, working code.

- Adding a third `Subject` subtype requires only a new class implementing `displaySubjectDetails()` / `getSubjectType()` — no existing subject code changes.
- `LetterGrade` and `GpaLetterGrade` are enums whose thresholds live as data on each constant; adding a new grading tier means adding one enum constant, not editing an if-chain.

OCP was violated, found, and fixed: `StudentSearcher.searchByType(boolean isHonors)` and `StatisticsCalculator.compareStudentTypes()` classified students via a `boolean` parameter and `instanceof HonorsStudent` checks — even though every `Student` already carried a `StudentType` enum internally, just never exposed publicly. Adding a third student type would have meant reworking a boolean parameter (which can only ever represent two states) and every `instanceof` check by hand. **Fix:** exposed `abstract StudentType Student.getType()`; both call sites now compare `getType()` against the enum instead. Adding a third type is now purely additive: one enum constant, one subclass.

4.3 Liskov Substitution Principle (LSP)

Definition: anywhere a base type is expected, any subtype must work correctly as a drop-in replacement, without surprising callers (no strengthened preconditions, no weakened postconditions, no new exceptions the base didn't promise).

Verified clean: `RegularStudent / HonorsStudent` and `CoreSubject / ElectiveSubject` both honor their base contracts exactly — no override throws an exception the base method doesn't declare, no override requires a caller to do anything extra first. Any code holding a `Student` or `Subject` reference works identically no matter which concrete subtype it actually holds.

4.4 Interface Segregation Principle (ISP)

Definition: no client should be forced to depend on methods it doesn't use — prefer several small, focused interfaces over one fat one.

Every interface here is small and single-purpose: `Calculable` (4 methods, all GPA-related), `Exportable` (2 methods), `Searchable` (4 methods, all search variants), `Gradable` (2 methods). No interface forces an implementer to stub out irrelevant methods.

4.5 Dependency Inversion Principle (DIP)

Definition: high-level modules shouldn't depend on low-level modules directly — both should depend on abstractions.

Every layer takes its collaborators as **constructor-injected interfaces**, never concrete classes:

```
public class StudentServiceImpl implements StudentService {  
    private final StudentRepository studentRepository; // interface, not StudentRepositoryImpl  
    public StudentServiceImpl(StudentRepository studentRepository) {  
        this.studentRepository = studentRepository;  
    }  
}
```

`Main` is the single, intentional exception — the **composition root**. Something in the program has to `new` the concrete implementations at least once; that's `Main`'s entire job, and only `Main` does it. Every other class is unaware which concrete implementation it was handed.

Part V — Java Language Mechanics (Behind the JVM Curtain)

This section is written for an *engineer*, not just "what does this keyword do" — it explains what the JVM actually does with each construct, since that's what distinguishes someone who memorized definitions from someone who understands the runtime.

5.1 JVM Runtime Data Areas — Where Things Actually Live

Area	What lives there	In this codebase
Method Area / Metaspace	Class metadata, method bytecode, <code>static</code> fields	<code>Student.studentCounter</code> , <code>Grade.gradeCounter</code> — one copy total, shared by every instance
Heap	Every object created with <code>new</code>	Every <code>Student</code> , <code>Grade</code> , <code>Subject</code> , <code>DTO</code> , exception instance
Stack (one per thread)	Local variables, method call frames, return addresses	Every local variable inside e.g. <code>GPACalculator.cumulativeGPA()</code>
PC Register	Address of the currently executing instruction, per thread	Irrelevant here — this app is single-threaded (one <code>main</code> thread only)

This app never creates a second thread, so there is exactly one call stack and no concurrency concerns to reason about — which matters for the `volatile` discussion below.

5.2 `static` — Class-Level State

A `static` field belongs to the *class*, not to any instance — there is exactly one copy, allocated once when the class is first loaded by the classloader, living in the Method Area for the lifetime of the JVM process. This codebase uses it for two auto-incrementing ID counters:

```
public abstract class Student {
    private static int studentCounter = 0; // ONE counter, shared by every Student ever created

    private String generateStudentId() {
        studentCounter++;
        return String.format("STU%03d", studentCounter);
    }
}
```

Every `new RegularStudent(...)` and every `new HonorsStudent(...)` increments the *same* counter, because both subclasses inherit the one static field from `Student` — there's no per-subclass copy. This is exactly why the test suite's own README warns: never assert an exact ID like `"STU001"`, because the counter is shared across every test class in the same JVM run (Surefire may run test classes in any order) — assert the *format* instead.

Why not an instance field instead? If `studentCounter` were an instance field, every new `Student` would start its own counter at 0 — `generateStudentId()` would always produce `"STU001"`, guaranteeing ID collisions. The whole point of `static` here is *shared* state across every instance.

5.3 `final` — Three Different Meanings

Applied to	Meaning	Example in this codebase
a variable/field	Can be assigned exactly once (reference cannot be resealed after that — though a mutable object it points to can still change internally)	<code>private final StudentRepository studentRepository;</code> — every repository/service field is <code>final</code> , assigned once in the constructor, guaranteeing it's never accidentally reassigned later
a method	Cannot be overridden by a subclass	<code>Student.getType()</code> is <code>final</code> — it always delegates to <code>getType().name()</code> , so no subclass can quietly reintroduce the duplicated-string-literal bug this replaced
a class	Cannot be subclassed at all	None declared explicitly here, but every DTO/utility class (<code>StudentDTO</code> , <code>DateFormats</code>) is implicitly "final in spirit" — they hold no behavior worth extending

5.4 `volatile` — Not Used Here, and Why That's Correct

This keyword doesn't appear anywhere in this codebase, and an engineer should be able to explain *why that's the right call*, not just what the keyword means:

`volatile` solves a **multi-threaded visibility** problem: without it, one thread's write to a shared field might sit in that thread's CPU cache and never become visible to another thread reading the same field, because the compiler/CPU are free to reorder and cache reads/writes for

performance under the Java Memory Model (JMM). Marking a field `volatile` forces every read to go to main memory and every write to flush to main memory immediately, establishing a happens-before relationship between threads.

This application has exactly one thread — the JVM's `main` thread running the console loop. There is no second thread ever created (no `Thread`, no `ExecutorService`, no async callback), so there is no visibility problem to solve. Marking `studentCounter` or `useRoleBased` `volatile` would add a small runtime cost (bypassing CPU caching) with zero benefit. If this app ever became a multi-user server handling concurrent requests, that calculus would change immediately — at that point, unsynchronized shared mutable state like `Student.studentCounter` would become a real race condition, and the fix would likely be `AtomicInteger` rather than a bare `volatile int`, since incrementing (`counter++`) is a read-modify-write that `volatile` alone does not make atomic.

5.5 `this` and `super`

- `this` — a reference to the current object instance; used here mainly in constructors to disambiguate a parameter name from a field of the same name (`this.studentId = studentId;`), and via `this(...)` to chain constructor overloads, e.g. `GradeServiceImpl`'s no-arg constructor delegates to its 3-argument constructor with default `Impl`s.
- `super` — refers to the parent class; used in every subclass constructor to initialize inherited fields (`super(name, age, email, phone);` in `RegularStudent`'s constructor calls `Student`'s constructor first, before any subclass-specific code runs). Constructor chaining via `super(...)` is mandatory — Java inserts an implicit `super()` call if you don't write one, and it must be the very first statement in a constructor.

5.6 `enum` — What the Compiler Actually Generates

Writing `public enum Role { TEACHER, STUDENT }` is syntactic sugar. The compiler generates a real class that implicitly extends `java.lang.Enum<Role>`, and each named constant becomes a `public static final Role` field — a genuine singleton instance, constructed exactly once when the class is first loaded, never again. That's why `==` is always safe (and preferred over `.equals()`) for comparing enum values — there is only ever one `Role.TEACHER` object in the entire JVM.

```
public enum GpaLetterGrade {
    A(93, 4.0, "A"), A_MINUS(90, 3.7, "A-"), /* ... */ F(0, 0.0, "F");

    private final double minPercentage, gpaPoints;
    private final String label;

    GpaLetterGrade(double minPercentage, double gpaPoints, String label) { // implicitly private
        this.minPercentage = minPercentage; this.gpaPoints = gpaPoints; this.label = label;
    }
}
```

Each enum constant can carry its own fields, exactly like an object — `GpaLetterGrade` uses this to keep percentage threshold, GPA points, and display label together as one source of truth, replacing three separately hand-maintained parallel arrays that had drifted apart from each other in an earlier version (the root cause of the project's headline bug, documented as KI-1 in `CHANGELOG.md`).

5.7 Interfaces, Dynamic Dispatch, and `invokevirtual`

When code calls `calculable.percentageToGPA(85)` where `calculable` is declared as the `Calculable` interface type, the compiler doesn't know which class's method will actually run — only `GPA Calculator` implements `Calculable` here, but in principle any class could. The JVM resolves this at runtime using the `invokeinterface` (or `invokevirtual` for abstract-class polymorphism) bytecode instruction: it looks up the object's actual runtime class, finds that class's method table (a per-class array of method pointers built at class-load time), and jumps to the correct method body. This lookup has a small, constant-time cost — noticeably more than a direct `invokestatic` call to a `static` method, but the JIT compiler can often inline it away once it observes the call site is monomorphic (always resolves to the same class) at runtime.

5.8 Exceptions — Stack Unwinding and `try-with-resources`

All 10 custom exceptions in this project extend `RuntimeException` (via the shared `ApplicationException` root) — they are **unchecked**, meaning the compiler does not force a caller to declare or catch them. When one is thrown (e.g. `new StudentNotFoundException(...)` inside `StudentRepositoryImpl.findStudentById()`), the JVM does not immediately search for a handler in that method — it **unwinds the call stack** frame by frame (repository → service → manager → console action → `ConsoleApp`'s loop), popping each method's local stack frame, running any `finally` blocks encountered along the way, until it finds a stack frame whose `try` block is covered by a matching `catch`. In this app that's always `ConsoleApp`'s central switch of `catch` clauses, ending in `catch (ApplicationException e)` — deliberately not `catch (Exception e)`, so a truly unexpected bug (a `NullPointerException`, say) is *not* silently swallowed and still crashes loudly, which is exactly what you want during development.

`try-with-resources` (used in `FileExporter` / `BulkImportService` for `FileWriter`, and in `CSVParser` for `Scanner`) compiles to an implicit `finally` block that calls `.close()` on the resource — guaranteed to run whether the `try` block completes normally, returns early, or throws. This requires the resource type to implement `java.lang.AutoCloseable`.

5.9 Generics and Type Erasure

`List<Student>`, `List<Grade>`, `List<String>` are used throughout. At compile time, the generic type parameter enforces that only a `Student` can go into a `List<Student>` — but at runtime, due to **type erasure**, that information is gone: the compiled bytecode just sees `List`, and the compiler inserts an invisible cast at every read site. This is why you cannot do `new T[10]` or check `if (list instanceof List<Student>)` in Java — the generic type parameter simply isn't present at runtime to check against.

5.10 Streams

Used in a few display-mapping spots, e.g. converting a `List<Student>` search result to a `List<StudentDTO>`:

```
List<StudentDTO> resultDtos = results.stream().map(StudentMapper::toDto).collect(Collectors.toList());
```

This builds a lazy pipeline (`stream()` does no work by itself), applies `toDto` to each element only when the terminal operation (`collect`) is invoked, producing a new list without mutating the original. `StudentMapper::toDto` is a **method reference** — shorthand for the lambda `s -> StudentMapper.toDto(s)`.

5.11 Garbage Collection — Why This App Never Frees Memory Manually

When `StudentRepositoryImpl.deleteStudent()` removes an entry by overwriting its array slot with `null` (after moving the last element into its place), the `Student` object that used to be there becomes **unreachable** — nothing in the program holds a reference to it anymore. The JVM's garbage collector will eventually reclaim that memory automatically; nothing in this codebase (or in Java generally, outside of rare off-heap scenarios) ever calls `free()` or manages memory by hand. This is a core reason Java was chosen for this kind of application over C/C++: no manual memory-management bugs (use-after-free, double-free) are possible.

Part VI — Logging Architecture

6.1 The `Logger` Class

```
public final class Logger {
    public enum Level { DEBUG, INFO, WARN, ERROR }
    private static volatile Level threshold = resolveThreshold();

    public static void debug(String message) { log(Level.DEBUG, message); }
    public static void info(String message) { log(Level.INFO, message); }
    public static void warn(String message) { log(Level.WARN, message); }
    public static void error(String message) { log(Level.ERROR, message); }

    private static void log(Level level, String message) {
        if (level.ordinal() < threshold.ordinal()) return; // below threshold - skip
        System.err.printf("[%s] %-5s %s\n", timestamp(), level, message);
    }
}
```

Here `volatile` IS used — on `Logger.threshold` — because `setLevel()` lets tests change the logging threshold at runtime, and while this app is single-threaded in practice, the field is written from test code and read from production code potentially across different execution contexts within the same JVM (parallel test execution is a real possibility with Surefire); marking it `volatile` is the cheap, correct insurance against that, unlike the counters in §5.4 which truly have no such exposure.

Level filtering: levels are ordered by their enum declaration order (`DEBUG < INFO < WARN < ERROR`), so `level.ordinal() < threshold.ordinal()` is a cheap integer comparison that skips building/printing a message below the configured threshold — default `INFO`, configurable via the JVM system property `-Dlog.level=DEBUG`, or programmatically via `Logger.setLevel(...)` in tests.

Why `System.err`, not `System.out`: the console menu's user-facing output (menus, results, confirmations) goes to `System.out`; every log line goes to `System.err`. This means a user (or a script) can redirect one stream without capturing the other — e.g. `java -cp target/classes Main 2>debug.log` keeps the menu clean on the terminal while every log line still lands in a file.

6.2 Why a Hand-Rolled Logger, Not Log4j/SLF4J

A real framework (SLF4J + Logback, or Log4j2) offers structured logging, log rotation, multiple appenders (file, console, network), and MDC (per-request context). None of that is needed for a single-threaded console app with no persistence and no concurrent requests — pulling in a full framework would be a dependency for a problem this app doesn't have. The hand-rolled `Logger` gives exactly the four levels and one output stream this app actually uses, with zero external dependencies. If this evolved into a server application, migrating to SLF4J's facade would be the very next thing to do — its `Logger` interface has almost the same four-method shape already, which would make that migration mechanical.

Part VII — Exception Handling Architecture

7.1 The Hierarchy

```
RuntimeException (java.lang)
├── ApplicationException (abstract – the ONE common root for everything this app throws on purpose)
│   ├── StudentNotFoundException (studentId, availableIds – lets ConsoleApp suggest valid IDs)
│   ├── StudentValidationException
│   ├── StudentException (studentId – general-purpose, e.g. repository storage full)
│   ├── SubjectNotFoundException
│   ├── SubjectValidationException
│   ├── SubjectException (subjectCode – general-purpose, e.g. repository storage full)
│   ├── GradeException (gradeId – general-purpose, e.g. repository storage full)
│   ├── InvalidGradeException (attemptedGrade – e.g. entered 150)
│   ├── ExportException (filePath)
│   ├── ImportException (filePath, successfulRows, failedRows)
│   └── CSVImportException (whole-file read failure, distinct from a single bad row)
```

Found and fixed during a dedicated audit: grepping every `throw new RuntimeException(...)` in `src` (excluding tests) turned up exactly two: `StudentRepositoryImpl.addStudent()` and `SubjectRepositoryImpl.addSubject()`, both for the identical "backing array is full" condition that `GradeRepositoryImpl.addGrade()` already handled correctly via `GradeException` - Grade had a general-purpose exception beyond just "not found"/"validation failed," but Student and Subject didn't, so those two call sites fell back to the generic root type instead. Fixed by adding `StudentException` / `SubjectException`, mirroring `GradeException`'s existing role exactly, and swapping both throw sites over. Every other `catch (RuntimeException e)` in the codebase turned out to be a different, legitimate pattern on inspection - either a deliberate "best-effort, swallow and continue" catch (the static-counter sync in `StudentManager` / `GradeManager`, which never crashes construction over a not-yet-available data source) or a "log for visibility, then rethrow the exact same specific exception unchanged" wrapper (`GradeServiceImpl.recordGrade()`) - neither converts anything to a generic type, so neither needed to change.

7.2 Why Everything Extends `RuntimeException` (Unchecked), Deliberately

Java offers *checked* exceptions (must be declared with `throws` or caught, enforced by the compiler) and *unchecked* ones (no such enforcement). This project chose unchecked deliberately: every custom exception here represents a business-rule violation (a bad student ID, an out-of-range grade) that the caller almost never wants to recover from locally — it wants to bubble all the way up to one central handler in `ConsoleApp`. Checked exceptions would force every intermediate layer (`StudentServiceImpl`, `StudentManager`, every `console.*Action`) to either declare `throws StudentNotFoundException` on every method signature or catch-and-throw pointlessly — pure boilerplate with no benefit, since there's nowhere useful to "handle" it before `ConsoleApp`.

7.3 No Generic `catch (Exception e)` — Why That Matters

`ConsoleApp`'s central handler ends in `catch (ApplicationException e)`, never `catch (Exception e)`. This is a direct, explicit requirement in the assignment brief ("No generic Exception catching"), and the reasoning is sound independent of the brief: a generic catch would also swallow a genuine bug — a `NullPointerException` from a real coding mistake — printing a vague error message and continuing as if nothing happened, hiding the bug instead of surfacing it. Catching only the app's own exception family lets anything truly unexpected crash loudly and visibly, which is far easier to debug.

7.4 Recovery Data Riding Along With the Exception

Several exceptions carry structured data beyond a message string specifically so the top-level handler can be more helpful than "something went wrong":

```
throw new StudentNotFoundException(
    "Student with ID '" + studentId + "' not found.",
    studentId,
    getAvailableStudentIds() // caught in ConsoleApp, printed as "Available student IDs: STU001, STU002, ..."
);
```

Part VIII — Class-by-Class Reference

Every class in the `src` tree (excluding `tests/`), grouped by package, with its purpose and every non-trivial method. Trivial one-line getters/setters are omitted from the method lists below for brevity — assume every private field has a corresponding public getter unless noted.

8.1 `model` — Domain Objects

Student `model.student.Student` (abstract)

Base type for every student. Holds ID, name, age, email, phone, status, and a transient grade list.

- `calculateAverageGrade(): double` — sums the transient `grades` list and divides by its size; returns 0.0 if empty (avoids division by zero).
- `isPassing(): boolean` — `calculateAverageGrade() >= getPassingGrade()`; delegates the threshold to the subclass.
- `abstract getType(): StudentType` — the OCP fix from Part IV; each subclass returns its own enum constant.
- `final getStudentType(): String` — `getType().name()`; cannot be overridden, so the two subclasses can no longer duplicate this logic independently.
- `abstract getPassingGrade(): double` / `abstract displayStudentDetails(): void` — subclass-specific.
- `static initializeCounter(int highestExistingSequence): void` — called once at startup by `StudentManager` to advance the static ID counter past whatever's already stored, so freshly-added students never collide with pre-seeded ones.

RegularStudent / HonorsStudent `model.student`

`RegularStudent`: passing grade 50% (`StudentType.REGULAR`). `HonorsStudent`: passing grade 60%, plus `checkHonorsEligibility(): boolean`, which recomputes (never caches) whether the student's current average clears their own passing threshold — recomputed on every call, not cached, so it's always current after a grade changes.

Subject (abstract) / CoreSubject / ElectiveSubject `model.subject`

Mirrors the `Student` hierarchy: `Subject` holds name + code; each subclass supplies its own `displaySubjectDetails()`, `getSubjectType()`, and an `isMandatory()` flag (true for `Core`, false for `Elective`).

Grade `model.grade.Grade` — implements `Gradable`

Immutable-in-spirit value once constructed: ID and date are auto-generated in the constructor and never change afterward.

- `Grade(String studentId, Subject subject, double gradeValue)` — validates the value via `recordGrade()` / `validateGrade()` (the `Gradable` contract) *before* assigning an ID or date; throws `InvalidGradeException` if out of range 0–100, so an invalid grade never gets an ID at all.
- `static reconstruct(gradeId, studentId, subject, gradeValue, date): Grade` — a private constructor exposed via a static factory, used only when rebuilding a grade already known-valid (e.g. from bulk import), skipping ID/date auto-generation.
- `getLetterGrade(): LetterGrade` — delegates to `LetterGrade.fromNumeric(grade)`.

Enums `model.enums`

- `Role { TEACHER, STUDENT }` — replaces a former raw `boolean isTeacher` flag.
- `StudentType { REGULAR(50.0), HONORS(60.0) }` — carries its own passing-grade threshold.
- `StudentStatus { ACTIVE, INACTIVE, SUSPENDED, GRADUATED }`.
- `SubjectType { CORE, ELECTIVE }`.
- `LetterGrade { A(85), B(70), C(55), D(40), F(0) }` with static `fromNumeric(double): LetterGrade` — iterates constants in declaration order, returns the first whose threshold the grade clears.
- `GpaLetterGrade` — 11 constants (A through F), each carrying percentage threshold, GPA points, *and* display label together; `fromPercentage()` and `fromGpaPoints()` static factories.

8.2 `repository` — Persistence Layer

StudentRepository / GradeRepository / SubjectRepository (interfaces) + their *Impl

Each interface defines CRUD-shaped operations (`addX`, `findXById` / `findXByCode`, `getAllX`, `deleteX`, plus `updateStudent`). Each `Impl` is backed by exactly one fixed-size array (`Student[50]`, `Grade[200]`, `Subject[50]`) plus a running count field. `StudentRepositoryImpl` and `SubjectRepositoryImpl` seed themselves with demo data in their constructors (5 students: 3 Regular + 2 Honors; 6 subjects: 3 Core + 3 Elective). `deleteX` methods swap the last live element into the deleted slot and null out the vacated last slot — an $O(1)$ removal that doesn't leave a gap, at the cost of not preserving insertion order after a delete.

8.3 service — Business Rules

StudentServiceImpl

`addStudent(Student): void` — runs `StudentValidator.validateStudent()` first (name/age/email/phone/ID format), logging and rethrowing on rejection; only then persists.

`getStudentById(String): Student` — throws `StudentNotFoundException` if the repository somehow returns `null` — defensive code for a state the repository's own contract already prevents, since `findStudentById` itself throws `StudentNotFoundException` on a miss. (This used to fall back to a plain `RuntimeException` instead of the declared exception type — found and fixed during a dedicated audit of every generic-exception throw site in the codebase; see Part VII.)

GradeServiceImpl

Three constructor overloads (telescoping pattern): no-arg (news its own default `Impl`s — a minor, isolated DIP compromise used only in one test), two-repository, and full three-repository. `recordGrade()` checks the student exists, then the subject exists, before persisting — the grade's own 0–100 range check already happened inside `Grade`'s constructor, so this method doesn't repeat it.

8.4 manager — Facades

StudentManager

`findStudent(String): Student (nullable)` — catches `StudentNotFoundException` internally and returns `null` instead, giving callers a simple null-check API rather than forcing every caller to catch an exception just to check existence.

`hydrateGrades(Student): void (private)` — the most important private method in the app: clears the student's transient grade list and refills it from `GradeManager`, since grades live in their own repository, not on the `Student` object. Called before every read path that needs an accurate average.

GradeManager

Wraps `GradeService` and `SubjectRepository`; adds grade-history formatting (`viewGradesByStudent`, sorted by date descending, ties broken by grade ID descending) and average calculations split by subject type (`calculateCoreAverage`, `calculateElectiveAverage`, `calculateOverallAverage`).

StudentSearcher implements Searchable

`searchById`, `searchByName` (case-insensitive substring match), `searchByGradeRange`, `searchByType(StudentType)` (fixed from a `boolean` — see Part IV). `getSearchDescription()` builds the human-readable label shown alongside results ("Name: \"John\"", "Type: Honors", etc.).

8.5 calculators

GPA Calculator implements Calculable

Every method delegates to `GpaLetterGrade` — no independent thresholds of its own anymore. `cumulativeGPA(studentId)` averages `percentageToGPA()` across every grade; `classRank(studentId)` counts how many other students have a strictly higher average, +1.

StatisticsCalculator

The largest class in the project (253 lines). `calculateDistribution()` buckets every grade by `LetterGrade.ordinal()`. `calculateStats()` computes mean, median (average of the two middle values for an even count), mode (ties broken toward the lowest value via a `TreeMap`'s ascending iteration order), standard deviation, and range, plus which student/subject earned the single highest and lowest grade. `compareStudentTypes()` now compares `getType()` instead of `instanceof HonorsStudent` (the same OCP fix as `StudentSearcher`).

8.6 `export` and `imports`

ReportGenerator implements Exportable

`exportSummary(studentId)` and `exportDetailed(studentId)` build report text as a `StringBuilder`; the detailed report maps each `Grade` to a `GradeDTO` via `GradeMapper` before formatting its grade-history table (the one DTO usage on the export side).

FileExporter

Wraps `java.io.FileWriter` directly — the one class in the project with no injected collaborator to mock, which is why it's the documented exception to the two-test-file convention (Part IX).

CSVParser

Parses a CSV, skipping (never aborting on) an invalid row — collects human-readable error strings instead. Validates, per row: exactly 4 columns, no empty field, a numeric grade in [0,100], a subject name that actually exists, and — the KI-10 fix — that the CSV's own declared `SubjectType` column actually matches the real subject's type (a row claiming "Elective" for a Core subject is rejected, where it used to import silently).

BulkImportService

Orchestrates: read file → `CSVParser.parse()` → for each valid row, look up the student (fail the row if unknown) → construct and persist a `Grade` → write a timestamped log file under `imports/` summarizing the whole run, success or partial failure. A log-write failure is caught and logged, never silently swallowed, and never rolls back grades already recorded (the import itself already happened by that point).

8.7 `console` — Presentation Layer, and `app` — Entry Point

10 classes under `console`, one per menu option, all implementing `MenuAction` (`getOptionNumber`, `getLabel`, `execute`, `isAuthorizedFor(Role)` default `true`, `terminatesLoop` default `false`). `ConsoleUtils` holds the two behaviors more than one action needs: `promptEnter(Scanner)` and `getAvailableStudentIds(StudentManager)`. Only `AddStudentAction`, `ViewStudentsAction`, `RecordGradeAction`, and `BulkImportAction` override `isAuthorizedFor` to require `Role.TEACHER` — every other action is readable by any role. These 10 classes have no automated tests of their own yet (0% in JaCoCo) — a known, deliberately scoped-out gap (Part XII), though each already takes its `Scanner` via constructor, so the same technique used for `ConsoleApp` below would work for each of them without further refactoring.

app.Main

The composition root, and nothing else. Builds every repository/service/manager/calculator exactly once as local variables, builds the `List<MenuAction>` in menu order, then hands both to a new `ConsoleApp` and calls `run()`. About 65 lines; deliberately too thin to need a test of its own.

app.ConsoleApp

The actual menu loop, extracted out of the old single-file `Main`: print menu (filtered by role) → read a number → find the matching action → check authorization → `execute()` inside a try/catch chain ending in `ApplicationException`. Takes its `Scanner` via constructor rather than binding to `System.in` in a `static final` field — that one change is what makes it unit-testable. Also had to move out of the default/unnamed package the old `Main` lived in: Java doesn't allow a class in a named package (like `tests.app`) to import a class from the unnamed package at all, so no test could ever have reached it there regardless of how testable its internals were. Covered by `tests/app/ConsoleAppTest.java` (scripted `Scanner` input, captured `System.out`, real `MenuAction` test doubles) and `tests/app/ConsoleAppMockitoTest.java` (mocked `MenuAction`s, verifying dispatch/authorization/retry interactions) — 17 tests covering every exception-translation branch, the role-based menu filter, invalid-input handling, and the retry-on-`InvalidGradeException` flow.

8.8 dto / mapper , utils

`StudentDTO` / `GradeDTO` are plain immutable data holders (final fields, constructor, getters, no behavior) used only by Search Students and the exported detailed report — Add Student/Record Grade still use real domain objects, since they need the full validation surface a DTO intentionally doesn't carry. `StudentMapper.toDto(Student): StudentDTO` and `GradeMapper.toDto(Grade): GradeDTO` are the only two mapping methods in the project — one direction only (see Part VIII's earlier discussion on why there's no `RequestDTO` / `ResponseDTO` split: there's no HTTP request/response boundary in a console app).

`utils.InputSanitizer.sanitize(String)` trims and strips control characters from raw console input. `utils.DateFormats` centralizes five named `SimpleDateFormat` patterns that used to be copy-pasted independently into four different classes. `utils.validators.StudentValidator` / `SubjectValidator` are stateless static validators throwing the appropriate `*ValidationException` on the first rule broken.

Part IX — Testing Strategy: JUnit 5, Mockito, and Alternatives

9.1 The Numbers

Metric	Value
Total tests	377, across 49 test files
JaCoCo instruction coverage, excluding only <code>console</code> (the 10 untested <code>*Action</code> classes)	92.3%
<code>calculators</code> package coverage	98.9% — clears the brief's "95%+ on critical business logic" bar
<code>exceptions</code> package coverage	95.6% (was 39.7% before a dedicated exception-hierarchy test pass — see Part VIII/CHANGELOG.md)
Overall instruction coverage (incl. untested <code>console</code>)	65.3%

Why overall coverage still looks lower than the business-logic figure: the 10 individual `console/*Action` classes (2,169 instructions, 0% covered by JUnit) each still read `Scanner` input and write `System.out` directly. This used to also be true of `Main` itself, but `Main`'s own menu-loop logic was extracted into `app.ConsoleApp` (Part VIII, §8.7) specifically to make it testable - proven out by 17 new tests covering it completely. The remaining gap is narrower and better understood now: each `*Action` already takes its `Scanner` via constructor, so the identical technique would work for each of them; it simply hasn't been done yet (Part XII). Excluding that one layer, coverage is 92.3%, comfortably above the 80% target - and every business-logic package individually clears 86%, most well above 95%.

9.1.1 Running JaCoCo and SonarQube

JaCoCo needs no separate setup — it attaches to the JVM automatically during the build:

```
mvn clean verify
```

Produces `target/site/jacoco/index.html` (a browsable per-package/per-class report) and `target/site/jacoco/jacoco.csv` (the same data, scriptable). The `report` execution is bound to Maven's `verify` phase, so `mvn test` alone does not regenerate it — `verify` (which runs `test` first anyway) does.

SonarQube needs a running server to send results to. Locally, on Windows, after downloading the free Community Edition:

```
# 1. Start the local server (first run initializes its own embedded DB - be patient)
bin\windows-x86-64\StartSonar.bat
# wait for "SonarQube is operational" in the log, then open http://localhost:9000
# (default admin/admin, forces a password change; create a project + token in the UI)

# 2. Run the analysis - clean verify first, in the SAME command, so Sonar's
# Java analyzer can read the JaCoCo execution data it needs for coverage:
mvn clean verify sonar:sonar -Dsonar.projectKey=<key> -Dsonar.token=<token>

# 3. Stop the server when done:
bin\windows-x86-64\StopSonar.bat
```

The `sonar-maven-plugin` is registered in `pom.xml` but bound to no lifecycle phase, so its mere presence never changes what `mvn test` / `mvn verify` do — `sonar:sonar` only runs when explicitly invoked.

9.2 The Two-File-Per-Class Convention

Nearly every production class has *two* test files:

- `<Class>Test.java` — behavioral tests using real collaborators (or, for service/manager layers, a hand-wired real dependency chain all the way down to the in-memory repositories). Answers: "does this produce the right output?"
- `<Class>MockitoTest.java` — the same class verified through *mocked* collaborators: interaction verification (did it call the repository exactly once?), call-order verification, and edge cases the real implementation can never actually produce but defensive code still guards against (a repository somehow returning `null`). Answers: "does this call its collaborators correctly, in the right order, the right number of times?"

Documented exceptions (no Mockito pair, because there's genuinely nothing to mock): `LetterGradeTest`, `GpaLetterGradeTest` (pure enum lookup functions), `InputSanitizerTest`, `DateFormatsTest` (pure static utilities), `StudentMapperTest`, `GradeMapperTest` (pure mapping functions over a real object, no branching to isolate), `FileExporterTest` (wraps `java.io.FileWriter` directly with no injected collaborator of its own).

9.3 JUnit 5 Lifecycle, and Every Annotation Actually Used

How JUnit 5 actually runs a test class (this is the part an engineer, not just a test-writer, should be able to explain): for *every single* `@Test` method, JUnit 5 creates a **brand-new instance** of the test class via reflection — this is the default `PER_METHOD` test instance lifecycle. That's why none of these 49 test files need to manually reset instance fields between tests: there is no shared state to reset, because no state is actually shared — each test method effectively gets its own object. (JUnit 5 also offers a `PER_CLASS` lifecycle via `@TestInstance(Lifecycle.PER_CLASS)`, which reuses one instance for every test in the class — not used anywhere in this suite, since nothing here needs the one scenario that justifies it: sharing an expensive piece of setup, like a real database connection, across many tests.)

Annotation	Count in this suite	What it does	Real example
<code>@Test</code>	233	Marks a method as a test case, discovered via reflection at runtime by the JUnit Jupiter engine	every test method
<code>@DisplayName</code>	222	A human-readable name shown in test reports/IDE output instead of the method name — lets the method name stay a valid identifier while the display name reads like a sentence	<code>@DisplayName("fromPercentage() matches the ReadMe-v2.md grading table")</code>
<code>@BeforeEach</code>	2	Runs immediately before <i>every</i> <code>@Test</code> method in the class, on that test's own fresh instance — used for setup shared by every test in that file	<code>GradeManagerTest.setUp()</code> / <code>StudentManagerTest.setUp()</code> build a fresh <code>StudentRepositoryImpl</code> before each test
<code>@AfterEach</code>	6	Runs after every test method, <i>whether it passed or failed</i> — the correct place for cleanup that must always happen (JUnit still runs it even if the test threw)	<code>BulkImportServiceTest.cleanup()</code> deletes the temp CSV/log files each test wrote
<code>@ParameterizedTest</code>	18	Runs the same test method body once per supplied argument set, instead of writing near-duplicate test methods by hand	<code>GpaLetterGradeTest</code> , <code>StudentValidatorTest</code>
<code>@CsvSource</code>	9	Supplies each row as inline CSV text - the source of a <code>@ParameterizedTest</code> 's arguments when a test needs more than one parameter per case	<code>GpaLetterGradeTest</code> 's <code>@CsvSource({"100, A", "93, A", "92, A_MINUS", ...})</code> turns 34 rows of the GPA grading table into a single test method
<code>@ValueSource</code>	9	Supplies a flat list of single values (all the same type: <code>strings</code> , <code>ints</code> , ...) - the simpler source when a test only needs one parameter per case	<code>invalid-name/invalid-email</code> sweeps in <code>StudentValidatorTest</code>

Not used, on purpose: `@BeforeAll` / `@AfterAll` (must be `static`; run once for the whole class - would matter for something genuinely expensive to set up once, like a real database connection, which nothing here has), `@Nested` (groups related tests into inner classes for readability in large test classes - these files are small/focused enough not to need it), `@Disabled` (temporarily skip a test - no test in this suite is disabled), `@Tag` / `@Timeout` / `@RepeatedTest` / `@TestMethodOrder` (test categorization, time limits, repetition, explicit ordering - none of which this suite's scope calls for, since every test is independent, fast, and order-agnostic by the `PER_METHOD` design above).

9.4 Every Assertion Method Actually Used

Assertion	Count	Checks
<code>assertEquals(expected, actual)</code>	222	The two values are equal (via <code>.equals()</code>), or numeric comparison with an optional delta for <code>double</code> s)
<code>assertTrue(condition)</code>	90	The boolean expression is <code>true</code>
<code>assertThrows(Class, executable)</code>	55	Running the given lambda throws an instance of exactly that exception type (or a subtype) - and returns the thrown exception so its message/fields can be asserted on too
<code>assertSame(expected, actual)</code>	12	The two references point at the <i>identical</i> object (<code>==</code>), not just an equal-looking one — used to confirm a wrapped cause/reference was passed through unchanged, not copied
<code>assertFalse(condition)</code>	11	The boolean expression is <code>false</code>
<code>assertDoesNotThrow(executable)</code>	10	Running the given lambda completes without throwing anything - used to positively confirm a valid case is accepted, the mirror image of <code>assertThrows</code>
<code>assertNull(actual)</code>	8	The value is <code>null</code>
<code>assertNotNull(actual)</code>	2	The value is not <code>null</code>
<code>assertNotEquals(unexpected, actual)</code>	2	The two values are <i>not</i> equal

Not used, on purpose: `assertAll(...)` - runs a group of assertions together and reports every failure at once instead of stopping at the first one; not used here because each test method is already written to check one specific behavior (matching its one-sentence `@DisplayName`), so there's rarely more than one or two closely-related assertions per test in the first place. `assertArrayEquals` / `assertIterableEquals` - not needed since no test compares raw arrays or needs iteration-order-sensitive collection equality beyond what `assertEquals` on a `List` already gives.

9.5 Mockito — Every API Actually Used, and How It Works Under the Hood

API	Count	What it does	Real example
<code>mock(Class)</code>	79	Creates a fake implementation whose every method returns a default ("smart null" - <code>0</code> / <code>false</code> / <code>null</code> / an empty collection) unless explicitly stubbed	<code>StudentManager studentManager = mock(StudentManager.class);</code>
<code>when(...).thenReturn(...)</code>	73 / 69	Stubs a specific call (matched by method + arguments) to return a specific value	<code>when(studentManager.findStudent("STU001")).thenReturn(student);</code>
<code>when(...).thenThrow(...)</code>	3	Same, but throws instead of returning	<code>when(studentService.getStudentById("NOPE")).thenThrow(new StudentNotFoundException("not found"));</code>
<code>verify(mock, times(n))</code>	102 / 45	Asserts a method was called exactly <code>n</code> times with the given arguments — proves <i>interaction</i> , not just outcome	<code>verify(studentManager, times(1)).findStudent("STU001");</code>
<code>verify(mock, never())</code>	35	<code>times(0)</code> — asserts a method was <i>never</i> called; a short-circuit check	confirms a rejected grade never reaches the repository
<code>verify(mock, atLeastOnce())</code>	10	Asserts a method was called one or more times, without caring about the exact count	authorization checks in <code>ConsoleAppMockitoTest</code>
<code>any()</code> / <code>anyString()</code>	10 / 1	Argument matchers meaning "match this call no matter what was passed here" - needed once <i>any</i> argument in a stub/verify call uses a matcher, since Mockito then requires every argument on that call to be a matcher too	<code>when(action.isAuthorizedFor(any())).thenReturn(true);</code>
<code>spy(realObject)</code>	6	Wraps a <i>real</i> object; every call runs the real method by default, but individual calls can still be verified or selectively overridden	<code>RegularStudentMockitoTest</code> spies on a real <code>RegularStudent</code> ; <code>isPassing()</code> genuinely delegates to <code>calculateAverageGrade()</code> / <code>getPassingGrade()</code> rather than logic
<code>doThrow(...).doNothing().when(mock).voidMethod()</code>	1 each	The required syntax for stubbing a void method (there's no return value to hang <code>when(...).thenReturn(...)</code> off of) - also sequences different behavior across successive calls: throw the first time, do nothing the second	<code>ConsoleAppMockitoTest</code> 's retry test: <code>doThrow(new InvalidGradeException(...)).doNothing().when(recordModels)</code> "fails once, then succeeds on retry"
<code>inOrder(mock1, mock2, ...)</code> / <code>InOrder</code>	3	Plain <code>verify()</code> only checks count, never relative order. <code>InOrder</code> additionally asserts the calls happened in a specific sequence <i>across one or more mocks</i> - needed whenever the order of operations is itself part of the behavior under test	<code>GradeServiceImplMockitoTest</code> : <code>inOrder.verify(students).findStudentById(...);</code> <code>inOrder.verify(subjects).findSubjectByCode(...);</code> <code>inOrder.verify(grades).addGrade(...);</code> - proves the stub the subject, before the grade is ever persisted

How `mock()` actually works: Mockito's default "inline" mock maker (used automatically since Mockito 5, which this project depends on) uses the `ByteBuddy` library plus the JVM's `java.lang.instrument` Instrumentation API to redefine the target class's bytecode at runtime, inserting an interceptor into every method. When a mock's method is called, that interceptor consults an internal invocation log: if a matching `when(...)` stub was registered, it returns/throws whatever was stubbed; otherwise it returns Mockito's default "smart null" and *records the call* so a later `verify(...)` can look it up. This is also exactly what makes mocking a `final` class or method possible without a separate tool like PowerMock — the older "subclass proxy" mock maker couldn't override `final` methods (you can't override `final` via normal subclassing), but rewriting the class's own bytecode directly has no such restriction.

A real version-compatibility bug hit during this project: `mockito-core:5.11.0` failed under JDK 25 with "Could not modify all classes" when mocking classes involving `Student`. Root cause: JDK 25 tightened restrictions on the exact bytecode instrumentation this inline mock maker relies on. Fixed by upgrading to `mockito-core:5.23.0`, a version with updated JDK-25 support. Good interview answer if asked "have you hit a real dependency/tooling bug?"

9.6 Testing Alternatives Not Used In This Codebase, and Why

Alternative	What it would do instead	Why it wasn't used here
<code>@Mock</code> / <code>@InjectMocks</code> / <code>@ExtendWith(MockitoExtension.class)</code>	The <i>annotation-driven</i> Mockito style: decorate test fields with <code>@Mock</code> , let the Mockito JUnit 5 extension instantiate and inject them automatically, and decorate the class-under-test field with <code>@InjectMocks</code> to have Mockito wire the mocks into its constructor for you	Every mock in this suite is instead created with an explicit <code>mock(Class.class)</code> call inside the test method body, and wired into the class-under-test by calling its real constructor directly. More typing per test, but every dependency a test relies on is visible in that one method - nothing is wired "by magic" via reflection you'd have to go looking for in a field declaration elsewhere in the file. A legitimate style choice, not a gap.
<code>ArgumentCaptor<T></code>	Captures the actual object passed into a mock during a call, so the test can inspect that object's internals <i>after</i> the call happened	Not needed here - every test already has the real object in hand before making the call (it constructs the <code>Grade / Student</code> itself), so there's nothing hidden inside the mock's invocation log left to capture and inspect afterward.
<code>verifyNoInteractions(mock)</code> / <code>verifyNoMoreInteractions(mock)</code>	Asserts a mock was never touched at all, or that no interactions happened beyond the ones already explicitly verified	The existing <code>never()</code> -based checks already assert "this specific method must not be called," which is the property every test in this suite actually cares about - not "nothing whatsoever happened to this mock."
<code>lenient()</code> stubbing	Suppresses Mockito's "unnecessary stubbing" strict-stubbing check for a specific <code>when(...)</code> that a test doesn't end up using on every path	Never needed - no test in this suite stubs a call it then doesn't exercise on some branch.

9.7 Framework-Level Alternatives Considered — Why Mockito

Alternative	Why not chosen
EasyMock	Older, more verbose "record-replay" API (must call <code>replay()</code> before use); Mockito's <code>when / verify</code> style is more readable and is the de facto Java standard today
PowerMock	Needed historically to mock <code>static / final / private</code> methods, but adds a custom classloader and is known to conflict with newer JDKs; Mockito's own inline mock maker (used here) now covers most of that need directly, without PowerMock's overhead
JMockit	Powerful but less widely adopted; smaller community, steeper learning curve for the same coverage Mockito already provides here
Hand-rolled fakes (no framework)	Possible, but reinvents <code>verify()</code> / call-counting / argument-matching by hand for every test — far more boilerplate for the same guarantee

Part X — Git & GitHub Workflow

10.1 Branching Strategy

Strict `main` / `develop` / `feature/*` flow, matching the brief's required strategy exactly:

- `main` — production-ready, protected; never touched directly during this entire project. Every fix and feature landed on `develop` first, with `main` gated for a later, separate authorization.
- `develop` — the integration branch; every feature branch merges here via `git merge --no-ff` (never a fast-forward, so the merge commit itself documents the feature as a discrete unit in history).
- `feature/*` — one branch per unit of work: `feature/exception-handling`, `feature/export-report`, `feature/gpa-calculation`, `feature/bulk-import`, `feature/class-statistics`, `feature/search-students`, `feature/ci-cd`, plus later `feature/BugFix-professional-structure`, `feature/BugFix-solid-principles`, `feature/BugFix-package-structure`, `feature/BugFix-student-read-only`, `feature/BugFix-menu-role-filter`, `feature/BugFix-pom-jacoco-placement`, and others — 16+ feature branches total across the project's life.

10.2 Commit Convention

Conventional Commits format throughout the later half of the project: `feat(scope): ...`, `fix(scope): ...`, `refactor(scope): ...`, `test(scope): ...`, `docs(scope): ...`. This was itself one of the 14 issues found in a requirements review (only ~11% of early commits followed the convention) and explicitly fixed going forward — a good example to cite if asked "how do you handle discovering your own past mistakes."

10.3 The Required Workflow, As Actually Practiced

1. Branch from `develop`
2. Implement + commit incrementally, with tests added alongside the same commit or a following one
3. Push the branch, let GitHub Actions CI run automatically
4. Only once CI is green, merge (`--no-ff`) into `develop` locally
5. Push `develop`, confirm CI is *also* green on `develop` post-merge (catches any merge-specific breakage a clean feature-branch CI run wouldn't)
6. `main` stays untouched pending separate, explicit authorization

10.4 CI/CD Pipeline

```
# .github/workflows/ci.yml (student-grade-management job)
on:
  push: {branches: [main, develop, 'feature/**']}
  pull_request: {branches: [main, develop]}
jobs:
  build-and-test-student-grade-management:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4 # Temurin JDK 17, Maven dependency cache
      - run: mvn -B test
```

Every push to any `feature/**` branch, and every push to `develop` / `main`, triggers an automatic full-suite run on a clean Ubuntu runner — meaning "works on my machine" is never enough; the suite must pass from a fresh checkout with no local state.

Part XI — Design Patterns Identified

Pattern	Where	Why it fits
Facade	<code>manager.StudentManager</code> , <code>manager.GradeManager</code>	Present one simple, higher-level API over several underlying services/repositories, hiding the coordination (hydrate grades, compute averages) from every caller above them
Strategy	<code>Calculable</code> (implemented by <code>GPACalculator</code>)	The GPA-conversion algorithm is swappable behind one interface — a different grading scale could be substituted without touching any caller
Composition Root	<code>Main</code>	All object construction and wiring happens in exactly one place, at startup — the pattern that makes constructor-based DIP (Part IV) practical without a DI framework like Spring
Data Transfer Object (DTO)	<code>dto.StudentDTO</code> , <code>dto.GradeDTO</code>	Projects a domain object down to only the fields a specific display path needs, decoupling console formatting from the full domain model
Repository	<code>repository.*</code>	Abstracts persistence behind a collection-like interface (<code>add</code> / <code>find</code> / <code>getAll</code> / <code>delete</code>), so the storage mechanism (arrays today) could change without touching any service
Static Factory Method	<code>Grade.reconstruct(...)</code> , <code>GpaLetterGrade.fromPercentage(...)</code>	A named static method instead of a public constructor, for a specific, named construction scenario distinct from “the normal way to make one of these”

Part XII — Known Limitations & Honest Discussion Points

An engineer being interviewed on this project should be able to name these unprompted — it demonstrates judgment, not just execution.

Limitation	Why it exists	What would change it
No persistence — all data lost on exit	Assignment brief explicitly required in-memory array storage	Swap each <code>*RepositoryImpl</code> 's array for a JDBC/JPA-backed implementation behind the same interface — no other layer would need to change, by design (DIP)
The 10 <code>console/*Action</code> classes have zero automated test coverage	Each still reads <code>Scanner</code> /writes <code>System.out</code> directly; <code>Main</code> 's own equivalent problem was already fixed by extracting <code>app.ConsoleApp</code> (constructor-injected <code>Scanner</code>), but the same extraction hasn't been applied to each individual action yet	Nothing architectural is missing - each action already takes its <code>Scanner</code> via constructor, so the identical scripted-input-plus-captured-output technique used for <code>ConsoleAppTest</code> would work today; it's just 10 more test files, not yet written
Single-threaded, single-user only	Console app; no server, no concurrent requests to serve	A move to a server (Spring Boot REST API, say) would require revisiting every shared-mutable-state assumption — the static ID counters, in particular, would need <code>AtomicInteger</code> or a database sequence
$O(n)$ linear-scan lookups in every repository	Required array-based storage; array sizes are small and fixed (200 grades max)	A <code>HashMap</code> -backed repository would make lookups $O(1)$, trivial to swap in behind the same interface
<code>getGrades()</code> exposes the live, mutable grade list	Convenience for <code>StudentManager.hydrateGrades()</code> , which deliberately clears and refills it	Return an unmodifiable/defensive copy and add a dedicated <code>replaceGrades(List<Double>)</code> method for the one legitimate internal caller

Part XIII — Anticipated Interview Questions (from the Assignment Rubric)

Drawn directly from `ReadMe-v2.md` 's grading rubric and testing scenarios — these are close to exactly what a reviewing engineer would ask, since the rubric is their checklist.

Q: Walk me through what happens, end to end, when I record a grade.

A: `console.RecordGradeAction.execute()` reads and sanitizes the student ID, subject choice, and grade value via `Scanner / InputSanitizer`, then calls `gradeManager.addGrade(grade)`. `GradeManager` delegates to `GradeService.recordGrade()`, which checks the student and subject both exist (throwing `StudentNotFoundException / SubjectNotFoundException` otherwise), then persists via `GradeRepository.addGrade()` into the `Grade[200]` array. The `Grade` object itself already validated its 0–100 range in its own constructor before any of this ran. Control returns up through the same call chain to `ConsoleApp` 's loop.

Q: Where exactly is SOLID actually demonstrated, not just claimed?

A: See Part IV in full — but the strongest answer is naming the two real violations that were *found and fixed* during a dedicated audit (the `Main` God Class for SRP, and the `boolean / instanceof`-based student-type dispatch for OCP), not just reciting where each principle theoretically applies.

Q: How is a Student prevented from adding, editing, or deleting anything?

A: Every write-capable `MenuAction` (`AddStudentAction`, `ViewStudentsAction`, `RecordGradeAction`, `BulkImportAction`) overrides `isAuthorizedFor(Role)` to require `TEACHER`; every read-only action accepts the interface's default `true`. Enforced twice: the menu itself is filtered by role before printing, and the dispatch loop re-checks authorization before calling `execute()`, so a Student typing an unlisted option number directly still fails safely.

Q: What's the difference between your two test files per class?

A: `<Class>Test` uses real collaborators end-to-end and checks output correctness. `<Class>MockitoTest` mocks the collaborators and verifies *interaction* — that the right method was called, the right number of times, in the right order — plus edge cases the real implementation can't produce but defensive code still guards.

Q: Why doesn't this project use checked exceptions?

A: Every custom exception represents a business-rule violation with nowhere useful to recover locally — it needs to bubble to one central handler in `ConsoleApp`. Checked exceptions would force every intermediate method signature to declare `throws`, pure boilerplate with no benefit here.

Q: How would this scale to multiple concurrent users?

A: Honestly — it currently wouldn't, safely. The static ID counters (`studentCounter`, `gradeCounter`) and the array-backed repositories have no synchronization; concurrent writes would race. Moving to a server model would require `AtomicInteger` counters (or a database sequence) and either synchronized access to the repositories or a real concurrent data store — this is flagged honestly in Part XII, not hidden.

Q: Why did you build a custom Logger instead of using SLF4J/Log4j?

A: A single-threaded console app with one output stream and four levels doesn't need log rotation, multiple appenders, or MDC — the full weight of a logging framework. The hand-rolled version gives exactly what's used, zero dependencies, and would be a mechanical swap to SLF4J's facade later if this became a server app.

Q: What would you do differently if you started over?

A: `Main` itself already went through exactly this — its menu loop was extracted into `app.ConsoleApp`, which takes its `Scanner` via constructor instead of binding to `System.in`, and is now fully covered by 17 tests. The same technique just hasn't been applied to each of the 10 individual `console/*Action` classes yet, even though each already takes a `Scanner` the same way — that's the one remaining, well-understood coverage gap in an otherwise 92%+-covered codebase.

Part XIV — Conclusion

This project took a flat, 9-class in-memory console app and rebuilt it into a layered, SOLID-compliant, dependency-injected, exception-safe, 92%+ business-logic-tested Java application, with a genuine Maven build, a genuine CI/CD pipeline, and a Git history that actually followed the required feature-branch discipline rather than just claiming to. Every architectural decision documented here — array storage, the composition root, colocated interfaces, the two-file test convention, the unchecked exception hierarchy — has a specific, defensible reason behind it, and every known gap (console test coverage, single-threaded assumptions, $O(n)$ lookups) is named rather than hidden. That combination — knowing not just *what* the code does but *why* it's shaped that way and *where it would need to change* — is what this report was built to put in front of a reviewing engineer.

Appendix A — The 14 Requirements-Review Issues (Debugging in Practice)

Before this codebase reached its current state, a full requirements review compared the actual `develop` code against `ReadMe-v2.md` line by line. Fourteen contradictions were found — each was reproduced first (often by literally running the buggy code and observing the wrong output), then fixed on its own feature branch with its own tests, then verified again. This is the single best material to cite for the rubric's "analyze complex bugs using systematic testing" criterion, because it's real, not hypothetical.

#	What was wrong	Root cause	Fix
KI-1	GPA letter grade was wrong for every tier except the very top and bottom (a 90% showed "A" instead of "A-")	Two independently hand-written if-chains (percentage→GPA and GPA→letter) had silently drifted out of sync with each other	Rewrote as one table-driven lookup (later an enum, <code>GpaLetterGrade</code>) — a single source of truth that cannot disagree with itself
KI-2	Exported summary report literally printed <code>"Name: [name]"</code> instead of the student's real name	<code>ReportGenerator</code> was never actually given a <code>Student</code> object, only a raw ID string, and had hardcoded a placeholder	Injected <code>StudentManager</code> so the class can resolve the real name itself from the ID
KI-3	Logging didn't cover <code>export / calculators / imports /search</code> ; one <code>IOException</code> was caught with no log call and no rethrow — a silent failure	Logging was added incrementally per-feature and some classes were simply missed	Added <code>Logger</code> calls throughout; the swallowed exception now logs via <code>Logger.error()</code> before continuing
KI-4	Zero test coverage on all 7 new v2 feature classes despite an explicit 80%+ requirement	Tests were never written alongside the features	Added a full <code>Test</code> + <code>MockitoTest</code> pair per class; suite grew from 210 to (eventually) 350 tests
KI-5	Two different, disagreeing letter-grade scales existed simultaneously (View Grade Report said "A" for 87%; View Class Statistics said "B" for the same 87%)	<code>StatisticsCalculator</code> hardcoded its own separate percentage bands instead of reusing <code>LetterGrade</code>	Statistics now buckets by <code>LetterGrade.fromNumeric(grade).ordinal()</code> directly — impossible to disagree with itself again
KI-6	"Mode" was documented and shown in the brief's own example output, but was never actually implemented	Simply missing	Added <code>calculateMode()</code> (ties broken toward the lower value)
KI-7	<code>Calculable</code> 's real method signatures didn't match the originally planned design (it took pre-assembled lists instead of resolving data itself by student ID)	Design drifted during implementation without updating the interface	<code>GPACalculator</code> now takes <code>GradeManager</code> / <code>StudentManager</code> via constructor injection and resolves everything internally by ID, matching the intended contract
KI-8	Custom exceptions extended <code>RuntimeException</code> directly (no shared base) and <code>Main</code> still had a generic <code>catch (Exception e)</code> — an explicit requirement violation	No common exception root existed yet	Introduced <code>ApplicationException</code> as the one abstract root; <code>Main</code> 's final catch is now <code>catch (ApplicationException e)</code>
KI-9	<code>StudentNotFoundException</code> 's recovery data (available IDs) was thrown away before <code>Main</code> ever saw it — every caller wrote its own duplicated "not found" message instead, one missing the ID list entirely	The exception was being caught early and converted to a plain <code>null</code> , discarding its payload	Every affected method now lets the exception bubble up intact to the one top-level handler that already knew how to print the available-IDs list
KI-10	Bulk import read the CSV's declared Subject Type column but never actually checked it — a row could claim "Elective" for a Core subject and import silently	The parsed value was discarded immediately after being read	<code>CSVParser</code> now rejects a row whose declared type disagrees with the subject's real type
KI-11	No <code>CHANGELOG.md</code> existed at all, despite being an explicit minimum requirement	Never created	Created — you're reading a report partly built from it
KI-12	No Maven/Gradle build existed; test dependencies were sourced from local jars by hand	Project started before a build tool was introduced	Added <code>pom.xml</code> and a matching CI job — see Part X
KI-13	Only ~11% of commits before this review used a conventional-commit prefix, despite it being required	Habit, not enforced	Every commit from this review onward uses <code>feat(scope): ... / fix(scope): ... /etc.</code>
KI-14	None of the 7 new v2 classes had JavaDoc; GPA/statistics thresholds were hardcoded inline with no named constants	Skipped under time pressure during initial implementation	Added class/method JavaDoc throughout; replaced inline numbers with named constants (later: enums)

Why this list matters for an interview: it proves the debugging skill the rubric asks for was actually exercised — each row is "I found a real discrepancy between spec and code, reproduced it, understood why it happened, fixed it on its own branch, and added a regression test so it can't silently return."

Appendix B — Real Console Transcripts

Actual output from running the compiled application (`java -cp target/classes Main`), not mocked-up examples — captured while verifying the role-based-access and bulk-import fixes described earlier in this report.

B.1 — Add Student, then Search by ID (Teacher role)

```
Enter choice: 1

ADD STUDENT
-----

Enter student name: John Doe
Enter student age: 20
Enter student email: john@example.com
Enter student phone: 1234567890

Student type:
1. Regular Student (Passing grade: 50%)
2. Honors Student (Passing grade: 60%, honors recognition)
Select type (1-2): 1
[INFO] Student added: STU006 (John Doe)

✓ Student added successfully!
Student ID: STU006
Name: John Doe
Type: REGULAR
Age: 20
Email: john@example.com
Passing Grade: 50%
Status: Active
```

B.2 — Bulk Import with a deliberately mixed CSV (every rejection type in one file)

```
Enter choice: 7

BULK IMPORT GRADES
-----

Enter filename (without extension): bulk_import_mixed_errors
Validating file... ✓
Processing grades...
[INFO] Parsed imports\bulk_import_mixed_errors.csv: 2 valid row(s), 6 error(s)
[INFO] Grade recorded: GRD001 (85.0) for student STU003

IMPORT SUMMARY
-----

Total Rows: 2
Successfully Imported: 1
Failed: 1

Failed Records:
Row 4: Unknown subject (History)
Row 5: Subject type mismatch - Mathematics is CORE, not ELECTIVE
Row 6: Grade out of range (150)
Row 7: Invalid grade number (abc)
Row 8: Invalid format (expected 4 columns, got 3)
Row 9: Unknown subject type (Advanced)
Row 3: Invalid student ID (STU999)

✓ Import completed!
1 grades added to system
See import_log_20260721_104943.txt for details
```

Every one of those seven failure lines corresponds to a distinct validation branch inside `CSVParser / BulkImportService` (Part VIII, §8.6) — this single file exercises every rejection path the importer has, in one run.

B.3 — Role-based menu filtering (Student role)

Enable role-based access control? (Y/N, default N): Y

Select your role:

1. Teacher

2. Student

Choose (1-2): 2

STUDENT GRADE MANAGEMENT - MAIN MENU

4. View Grade Report

5. Export Grade Report

6. Calculate Student GPA

8. View Class Statistics

9. Search Students

10. Exit

Role: Student

Enter choice:

Options 1, 2, 3, and 7 (every write-capable action) are simply absent from this menu — not listed-then-denied. Typing **7** directly anyway still produces "Access denied. This action is not available for your role." from the dispatch-time check, confirming defense-in-depth.

Appendix C — Glossary (Quick-Reference Definitions)

Term	Definition
JVM	Java Virtual Machine — the runtime that loads compiled <code>.class</code> bytecode and executes it, providing memory management (heap, GC), a call stack per thread, and platform independence
Metaspace / Method Area	Where class metadata and <code>static</code> fields live — one copy per class, for the life of the JVM process
Heap	Where every object created with <code>new</code> lives; managed by the garbage collector
Stack frame	One method call's local variables + return address; popped when the method returns or an exception unwinds past it
Bytecode	The platform-independent instruction set the <code>javac</code> compiler emits and the JVM executes (e.g. <code>invokevirtual</code> , <code>invokeinterface</code>)
Dynamic dispatch	Resolving which method body actually runs based on the object's real runtime class, not its declared reference type — the mechanism behind polymorphism
Type erasure	Generic type parameters (<code><T></code>) exist only at compile time; the compiled bytecode has no record of them
Checked vs. unchecked exception	Checked: compiler forces a <code>throws</code> declaration or a catch. Unchecked (extends <code>RuntimeException</code>): no such enforcement — used exclusively in this project
Happens-before	A Java Memory Model guarantee that one thread's write becomes visible to another thread's subsequent read — what <code>volatile</code> / <code>locks</code> / <code>synchronized</code> establish
Composition root	The single place in an application where the full object graph is constructed and wired together — here, <code>Main</code>
Mock vs. spy	A mock replaces a collaborator entirely (every method is fake unless stubbed); a spy wraps a real object (every method is real unless overridden) — both from Mockito
CI/CD	Continuous Integration / Continuous Delivery — automatically building and testing every push (here: GitHub Actions running <code>mvn test</code>) so integration problems surface immediately, not at release time